CRUD Agenda App

Guía completa paso a paso

Reflex | SQLAlchemy | Pydantic | SQLite

Explicación detallada de cada archivo del proyecto,
de cómo se conectan las capas y de cómo viaja la información
desde el botón que pulsa el usuario hasta la base de datos.

Documento generado a partir del código fuente de crud_agenda_app.zip

Contenido

Contenido 2

1. Qué es esta aplicación 3

2. Arquitectura en capas 4

3. Estructura del proyecto 5

4. Construcción paso a paso 6

Paso 1: Preparar el entorno y las dependencias 6

Paso 2: Configuración de la aplicación (config.py) 7

Paso 3: Configuración de Reflex (rxconfig.py) 8

Paso 4: Conexión a la base de datos (db/base.py) 8

Paso 5: El modelo de datos (db/models/task.py) 9

Paso 6: Esquemas Pydantic o DTO (db/schemas/task.py) 10

Paso 7: Servicio CRUD (services/task_service.py) 11

Paso 8: El estado de Reflex (states/task_state.py) 12

Paso 9: La interfaz de usuario (pages/index.py) 14

Paso 10: Punto de entrada de la aplicación (crud_agenda_app.py) 17

Paso 11: Ejecutar y probar la aplicación 18

5. Cómo viaja la información: flujo de cada operación 19

6. Observaciones y mejoras recomendadas 20

7. Glosario rápido 21

1. Qué es esta aplicación

CRUD Agenda App es una agenda de tareas web. Un **CRUD** es el conjunto de las cuatro operaciones básicas sobre datos: **C**reate (crear), **R**ead (leer), **U**pdate (actualizar) y **D**elete (eliminar). Con esta aplicación puedes:

- Ver la lista de tareas, ordenadas de la más reciente a la más antigua.
- Crear una tarea nueva con título, descripción, prioridad y fecha límite opcional.
- Editar cualquier tarea existente.
- Marcar una tarea como hecha o pendiente con una casilla (el título aparece tachado).
- Eliminar una tarea.

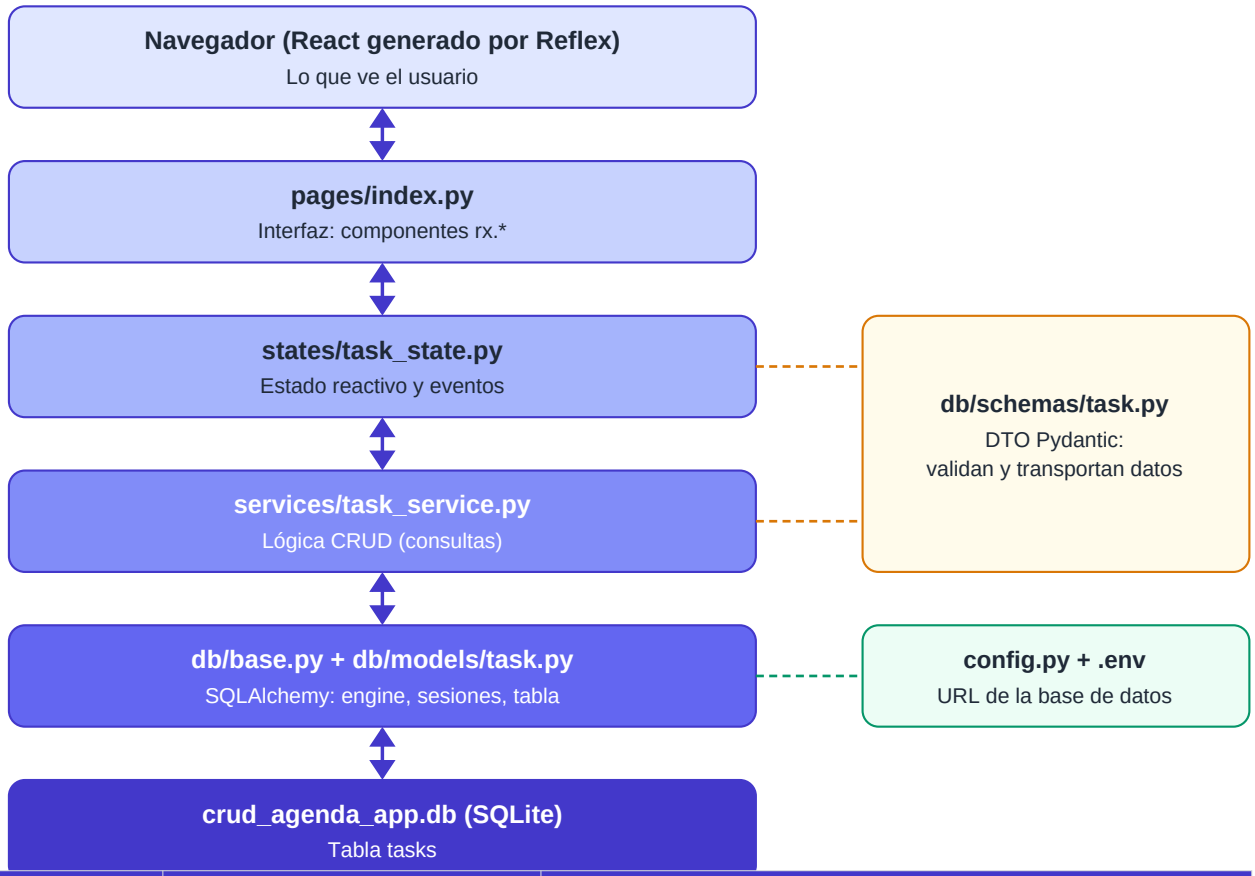
Todo está escrito **únicamente en Python**: la interfaz, la lógica y el acceso a la base de datos. No hay HTML, CSS ni JavaScript escritos a mano. Eso es posible gracias a **Reflex**, un framework que convierte los componentes de Python en una aplicación React para el navegador y ejecuta la lógica en un servidor Python que se comunica con el navegador en tiempo real.

Tecnologías utilizadas

Tecnología	Versión (pyproject.toml)	Para qué se usa
Python	>= 3.14	Lenguaje de todo el proyecto.
Reflex	>= 0.10.1	Framework web: interfaz, estado y servidor.
SQLAlchemy	>= 2.1.8	ORM: convierte clases Python en tablas y consultas SQL.
Pydantic	>= 2.14.3	Validación de datos y objetos de transferencia (DTO).
pydantic-settings	>= 2.16.2	Lectura de configuración desde variables de entorno y .env.
SQLite	incluido en Python	Base de datos en un único archivo: crud_agenda_app.db.
uv	-	Gestor de paquetes y entornos (archivo uv.lock).

2. Arquitectura en capas

El proyecto separa las responsabilidades en capas. Cada capa solo habla con la que tiene justo debajo, de modo que se puede cambiar una sin romper las demás (por ejemplo, cambiar SQLite por PostgreSQL solo afecta a la configuración y a la capa de base de datos).



Capa	Archivo	Responsabilidad
Interfaz	pages/index.py	Dibuja la pantalla y conecta cada botón con un evento del estado.
Estado	states/task_state.py	Guarda los datos que la pantalla muestra y define qué ocurre al hacer clic.
Servicio	services/task_service.py	Contiene las operaciones CRUD. No sabe nada de Reflex ni de la pantalla.
Esquemas	db/schemas/task.py	Define qué datos son válidos (DTO) y cómo viajan entre capas.
Modelo	db/models/task.py	Describe la tabla tasks como una clase Python.
Base de datos	db/base.py	Crea el engine, las sesiones y las tablas.
Configuración	config.py, .env, rxconfig.py	Parámetros: URL de la base de datos, entorno, claves.

Idea clave: por qué existen los DTO
El estado de Reflex necesita datos que pueda convertir a JSON para enviarlos al navegador. Un objeto ORM de SQLAlchemy está ligado a una sesión de base de datos y no sirve para eso. Por ello el servicio convierte cada fila **Task** en un **TaskDTO** (un modelo Pydantic simple) antes de entregarlo al estado.

3. Estructura del proyecto

```
crud_agenda_app/                    <- raíz del proyecto
|-- pyproject.toml                  <- dependencias y metadatos
|-- uv.lock                         <- versiones exactas instaladas
|-- rxconfig.py                     <- configuración de Reflex
|-- config.py                       <- ajustes de la aplicación (pydantic-settings)
|-- .env                            <- variables de entorno (vacío por ahora)
|-- crud_agenda_app.db              <- base de datos SQLite (se crea sola)
|-- assets/favicon.ico              <- icono de la pestaña
|-- AGENTS.md / CLAUDE.md           <- instrucciones para agentes de IA (Reflex)
|
|-- crud_agenda_app/                <- paquete principal de la aplicación
|   |-- crud_agenda_app.py          <- punto de entrada: crea la app y la página
|   |-- pages/index.py              <- interfaz (capa de presentación)
|   |-- states/task_state.py        <- estado y eventos de Reflex
|   |-- services/task_service.py    <- operaciones CRUD
|   |-- db/
|       |-- base.py                 <- engine, SessionLocal, Base, init_db()
|       |-- models/task.py          <- modelo SQLAlchemy Task
|       |-- schemas/task.py         <- DTO Pydantic
|
|-- .web/        <- generado por Reflex (frontend React). No se edita.
|-- .states/     <- generado por Reflex (estado en desarrollo). No se edita.
|-- reflex.lock/ <- generado por Reflex. No se edita.
|-- src/crud_agenda_app/            <- carpeta vacía sobrante de la plantilla
```

Las carpetas generadas (**.web**, **.states**, **reflex.lock**) las crea Reflex automáticamente al ejecutar la aplicación; no hace falta escribirlas ni versionarlas. El código que tú escribes está en **config.py**, **rxconfig.py** y la carpeta **crud_agenda_app/**.

4. Construcción paso a paso

Los pasos siguen el orden recomendado para construir la aplicación: primero los cimientos (entorno, configuración, base de datos), luego la lógica, y al final la interfaz. En cada paso se muestra el código completo del archivo y después se explica.

Paso 1: Preparar el entorno y las dependencias

El proyecto usa **uv** para gestionar el entorno virtual y las dependencias. Los comandos habituales para partir de cero son:

```
uv init                      # crea pyproject.toml
uv add reflex sqlalchemy pydantic pydantic-settings
uv run reflex init           # crea la estructura base de Reflex (.web, assets, rxconfig.py)
```

Archivo: `pyproject.toml`

```
[project]
name = "crud-agenda-app"
version = "0.1.0"
description = "Add your description here"
authors = [
    { name = "Tim", email = "dm161272@gmail.com" }
]
requires-python = ">=3.14"
dependencies = [
    "pydantic>=2.14.3",
    "pydantic-settings>=2.16.2",
    "reflex>=0.10.1",
    "sqlalchemy>=2.1.8",
]

[project.scripts]
crud-agenda-app = "crud_agenda_app:main"

[build-system]
requires = ["uv_build>=0.12.24,<0.13.0"]
build-backend = "uv_build"
```

- `requires-python = ">=3.14"`: exige Python 3.14 o superior. Si tienes una versión anterior, uv puede descargar una compatible.
- `dependencies`: las cuatro librerías que usa el código. Reflex trae consigo el servidor y las herramientas de compilación del frontend.
- `[project.scripts]`: declara un comando `crud-agenda-app` que apunta a `crud_agenda_app:main`. Ver la sección de observaciones: esa función **main** no existe todavía.

Paso 2: Configuración de la aplicación (config.py)

Centralizar los ajustes en un solo lugar evita escribir la URL de la base de datos o las claves directamente en el código. **pydantic-settings** los lee de variables de entorno o del archivo **.env** y los valida.

Archivo: config.py

```
from functools import lru_cache
from pathlib import Path

from pydantic import SecretStr, model_validator
from pydantic_settings import BaseSettings, SettingsConfigDict

ROOT = Path(__file__).resolve().parent.parent


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=ROOT / ".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

    env: str = "dev"
    app_name: str = "crud_agenda_app"
    debug: bool = False

    database_url: str = "sqlite:///crud_agenda_app.db"
    secret_key: SecretStr | None = None
    openai_api_key: SecretStr | None = None

    # only for Reflex itself
    api_url: str = "http://localhost:8000"


    @model_validator(mode="after")
    def _require_key_in_prod(self):
        if self.env == "prod" and self.secret_key is None:
            raise ValueError("SECRET_KEY is required in production")
        return self


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

- `class Settings(BaseSettings)`: cada atributo es un ajuste. Pydantic busca una variable de entorno con el mismo nombre (sin distinguir mayúsculas): `DATABASE_URL` rellena `database_url`.
- `SettingsConfigDict(env_file=..., extra="ignore")`: indica dónde está el .env y que las variables desconocidas se ignoren en lugar de dar error.
- `database_url`: por defecto `sqlite:///crud_agenda_app.db`, un archivo en la carpeta desde la que se ejecuta la app.
- `SecretStr`: tipo para secretos. Al imprimirse muestra asteriscos; el valor real se obtiene con `.get_secret_value()`. `openai_api_key` está declarada pero aún no se usa.
- `@model_validator(mode="after")`: regla de negocio. Si `env` es `"prod"` y no hay `secret_key`, la aplicación se niega a arrancar.
- `@lru_cache` sobre `get_settings()`: la configuración se crea una sola vez y se reutiliza (patrón singleton).
- `api_url`: dirección donde el navegador encuentra el servidor de Reflex (por defecto `http://localhost:8000`).

Ejemplo de contenido para el archivo **.env** (actualmente está vacío):

```
ENV=dev
```

```
DEBUG=true
DATABASE_URL=sqlite:///crud_agenda_app.db
# SECRET_KEY=cambia-esto-en-produccion
```

Paso 3: Configuración de Reflex (rxconfig.py)

Archivo: rxconfig.py

```
import reflex as rx
from config import get_settings

settings = get_settings()

config = rx.Config(
    app_name=settings.app_name,
    db_url=settings.database_url,
    api_url=settings.api_url,
    plugins=[
        rx.plugins.SitemapPlugin(),
        rx.plugins.TailwindV4Plugin(),
        rx.plugins.RadixThemesPlugin(),
    ]
)
```

- Reflex lee este archivo al arrancar. `rx.Config` recibe los parámetros del framework.
- `app_name`: debe coincidir con el paquete que contiene la app (**crud_agenda_app**). Reflex busca dentro de él el archivo `crud_agenda_app.py` y la variable `app`.
- `db_url` y `api_url`: se toman de `Settings`, así hay una única fuente de verdad.
- `plugins`: activa el mapa del sitio (Sitemap), Tailwind CSS v4 y los componentes Radix Themes (botones, tarjetas, diálogos, etc.).

> **Nota**
> Este archivo hace **from config import get_settings**, por eso Reflex debe ejecutarse desde la carpeta raíz del proyecto, donde está config.py.

Paso 4: Conexión a la base de datos (db/base.py)

Archivo: crud_agenda_app/db/base.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from config import get_settings

engine = create_engine(get_settings().database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(engine, expire_on_commit=False)


class Base(DeclarativeBase):
    pass


def init_db() -> None:
    """Prototype only. Switch to Alembic once the schema changes."""
    from crud_agenda_app.db import models  # noqa: F401  (registers tables)
    Base.metadata.create_all(engine)
```

- `create_engine(url, pool_pre_ping=True)`: el **engine** es el punto de conexión con la base de datos. `pool_pre_ping` comprueba que la conexión sigue viva antes de usarla.

- `sessionmaker(engine, expire_on_commit=False)`: fábrica de **sesiones**. Una sesión agrupa operaciones y las confirma juntas. Con `expire_on_commit=False` los objetos conservan sus datos después de guardar, algo necesario para convertirlos a DTO tras el commit.
- `class Base(DeclarativeBase)`: clase madre de todos los modelos. Registra las tablas en `Base.metadata`.
- `init_db()`: importa los modelos (para que queden registrados) y ejecuta `create_all`, que crea las tablas que aún no existen. Es una solución de prototipo; en un proyecto real se sustituye por migraciones con **Alembic**.

Paso 5: El modelo de datos (db/models/task.py)

Archivo: crud_agenda_app/db/models/task.py

```
from datetime import date, datetime

from sqlalchemy import String, Text, func
from sqlalchemy.orm import Mapped, mapped_column

from crud_agenda_app.db.base import Base


class Task(Base):
    __tablename__ = "tasks"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    description: Mapped[str] = mapped_column(Text, default="")
    priority: Mapped[str] = mapped_column(String(10), default="medium")
    due_date: Mapped[date | None]
    done: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
```

Cada atributo anotado con **Mapped[...]** es una columna. El tipo de Python decide si acepta nulos: `Mapped[str]` es obligatorio (NOT NULL) y `Mapped[date | None]` es opcional.

Columna	Tipo SQL	Valor por defecto	Descripción
id	INTEGER (clave primaria)	autoincremental	Identificador único de la tarea.
title	VARCHAR(200), NOT NULL	-	Título, máximo 200 caracteres.
description	TEXT, NOT NULL	"" (vacío)	Descripción libre.
priority	VARCHAR(10), NOT NULL	"medium"	low, medium o high.
due_date	DATE, admite NULL	NULL	Fecha límite opcional.
done	BOOLEAN, NOT NULL	False	Indica si la tarea está completada.
created_at	DATETIME, NOT NULL	CURRENT_TIMESTAMP	Fecha de creación, la pone la base de datos.

default frente a server_default
default=... lo aplica SQLAlchemy desde Python al insertar. **server_default=func.now()** lo aplica la propia base de datos (CURRENT_TIMESTAMP). Este esquema coincide exactamente con la tabla que existe en el archivo crud_agenda_app.db incluido en el proyecto.

Paso 6: Esquemas Pydantic o DTO (db/schemas/task.py)

Archivo: crud_agenda_app/db/schemas/task.py

```
from datetime import date, datetime
from enum import Enum

from pydantic import BaseModel, ConfigDict, Field


class Priority(str, Enum):
    low = "low"
    medium = "medium"
    high = "high"


class TaskCreateDTO(BaseModel):
    model_config = ConfigDict(use_enum_values=True)  # store "high", not Priority.high

    title: str = Field(min_length=1, max_length=200)
    description: str = ""
    priority: Priority = Priority.medium
    due_date: date | None = None


class TaskUpdateDTO(BaseModel):
    model_config = ConfigDict(use_enum_values=True)

    title: str | None = Field(default=None, min_length=1, max_length=200)
    description: str | None = None
    priority: Priority | None = None
    due_date: date | None = None
    done: bool | None = None


class TaskDTO(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    title: str
    description: str
    priority: str          # plain str is easier in the frontend than an Enum
    due_date: date | None
    done: bool
    created_at: datetime
```

Se definen tres DTO, cada uno para un caso distinto:

Clase	Uso	Detalles
Priority	Enumeración	Limita la prioridad a low, medium o high. Cualquier otro valor da error.
TaskCreateDTO	Entrada al crear	title obligatorio (1 a 200 caracteres). El resto tiene valor por defecto.
TaskUpdateDTO	Entrada al editar	Todos los campos son opcionales: solo se modifican los que se envían.
TaskDTO	Salida hacia la interfaz	Representa una tarea completa; se construye a partir del objeto ORM.

- `use_enum_values=True`: al validar, el enum se convierte a su texto ("high") en vez de dejar `Priority.high`, y así se guarda directamente en la columna VARCHAR.
- `Field(min_length=1, max_length=200)`: rechaza títulos vacíos o demasiado largos. El mensaje de error se mostrará en el diálogo.
- `from_attributes=True` en TaskDTO: permite crear el DTO leyendo atributos de otro objeto (`TaskDTO.model_validate(task_orm)`).

- En `TaskDTO`, `priority` es un `str` y no un enum: el comentario del código indica que resulta más sencillo para el frontend.

Paso 7: Servicio CRUD (services/task_service.py)

Archivo: `crud_agenda_app/services/task_service.py`

```
from sqlalchemy import select

from crud_agenda_app.db.base import SessionLocal
from crud_agenda_app.db.models.task import Task
from crud_agenda_app.db.schemas.task import TaskCreateDTO, TaskDTO, TaskUpdateDTO


def _to_dto(t: Task) -> TaskDTO:
    return TaskDTO.model_validate(t)


def list_tasks() -> list[TaskDTO]:
    with SessionLocal() as s:
        rows = s.scalars(select(Task).order_by(Task.created_at.desc())).all()
        return [_to_dto(r) for r in rows]


def create_task(dto: TaskCreateDTO) -> TaskDTO:
    with SessionLocal.begin() as s:
        task = Task(**dto.model_dump())   # dates stay date objects, enum already a str
        s.add(task)
        s.flush()
        s.refresh(task)
        return _to_dto(task)


def update_task(task_id: int, dto: TaskUpdateDTO) -> TaskDTO:
    with SessionLocal.begin() as s:
        task = s.get(Task, task_id)
        if task is None:
            raise LookupError(f"Task {task_id} not found")
        for key, value in dto.model_dump(exclude_unset=True).items():
            setattr(task, key, value)
        s.flush()
        s.refresh(task)
        return _to_dto(task)


def delete_task(task_id: int) -> None:
    with SessionLocal.begin() as s:
        task = s.get(Task, task_id)
        if task is not None:
            s.delete(task)
```

Aquí vive toda la lógica de acceso a datos. Fíjate en los dos modos de abrir una sesión:

- `with SessionLocal() as s`: sesión de **solo lectura**. Se cierra al salir del bloque.
- `with SessionLocal.begin() as s`: sesión **con transacción**. Si el bloque termina bien hace **commit** automático; si lanza una excepción hace **rollback** y no se guarda nada.

Función	Qué hace	SQL equivalente
list_tasks()	Devuelve todas las tareas como lista de TaskDTO, la más nueva primero.	SELECT * FROM tasks ORDER BY created_at DESC
create_task(dto)	Crea un Task con dto.model_dump(), hace flush() para obtener el id y refresh() para leer created_at, y lo devuelve como DTO.	INSERT INTO tasks (...) VALUES (...)

Función	Qué hace	SQL equivalente
update_task(id, dto)	Busca la tarea con s.get(); si no existe lanza LookupError. Aplica solo los campos enviados gracias a model_dump(exclude_unset=True).	UPDATE tasks SET ... WHERE id = ?
delete_task(id)	Busca la tarea y la elimina. Si no existe no hace nada.	DELETE FROM tasks WHERE id = ?

exclude_unset: el detalle más importante de update_task
Si el DTO es **TaskUpdateDTO(done=True)**, solo el campo **done** cuenta como "enviado", y por eso la casilla de la lista no pisa el título ni la descripción. En cambio, el formulario de edición envía los cuatro campos, de modo que una fecha vacía (None) **borra** la fecha límite guardada.

- `_to_dto` se llama **dentro** del bloque with, mientras la sesión sigue abierta, para poder leer los atributos del objeto ORM.
- Este módulo no importa nada de Reflex: se podría reutilizar desde una API REST o desde pruebas unitarias.

Paso 8: El estado de Reflex (states/task_state.py)

Archivo: crud_agenda_app/states/task_state.py

```
import reflex as rx
from pydantic import ValidationError

from crud_agenda_app.db.schemas.task import TaskCreateDTO, TaskDTO, TaskUpdateDTO
from crud_agenda_app.services import task_service


class TaskState(rx.State):
    tasks: list[TaskDTO] = []
    error: str = ""

    dialog_open: bool = False
    editing_id: int = 0
    form_title: str = ""
    form_description: str = ""
    form_priority: str = "medium"
    form_due_date: str = ""

    @rx.event
    def load(self):
        self.tasks = task_service.list_tasks()

    @rx.event
    def set_dialog_open(self, value: bool):
        self.dialog_open = value
        if not value:
            self.error = ""

    @rx.event
    def open_new(self):
        self.editing_id = 0
        self.form_title = ""
        self.form_description = ""
        self.form_priority = "medium"
        self.form_due_date = ""
        self.error = ""
        self.dialog_open = True

    @rx.event
    def open_edit(self, task_id: int):
        task = next((t for t in self.tasks if t.id == task_id), None)
        if task is None:
```

```
            return
        self.editing_id = task.id
        self.form_title = task.title
        self.form_description = task.description
        self.form_priority = task.priority
        self.form_due_date = task.due_date
        self.error = ""
        self.dialog_open = True

    @rx.event
    def save(self, form: dict):
        data = dict(
            title=form.get("title", "").strip(),
            description=form.get("description", "").strip(),
            priority=form.get("priority", "medium"),
            due_date=form.get("due_date") or None,
        )
        try:
            if self.editing_id:
                task_service.update_task(self.editing_id, TaskUpdateDTO(**data))
            else:
                task_service.create_task(TaskCreateDTO(**data))
        except ValidationError as e:
            self.error = e.errors()[0]["msg"]
            return
        self.error = ""
        self.dialog_open = False
        self.tasks = task_service.list_tasks()

    @rx.event
    def toggle(self, task_id: int, done: bool):
        task_service.update_task(task_id, TaskUpdateDTO(done=done))
        self.tasks = task_service.list_tasks()

    @rx.event
    def delete(self, task_id: int):
        task_service.delete_task(task_id)
        self.tasks = task_service.list_tasks()
```

Cómo funciona el estado. Una clase que hereda de **rx.State** contiene variables tipadas. Cuando un **evento** (método con **@rx.event**) modifica una variable con **self.variable = ...**, Reflex detecta el cambio y envía al navegador solo lo que cambió, y la interfaz se redibuja sola. No hay que escribir código de actualización manual.

Variables del estado

Variable	Tipo	Función
tasks	list[TaskDTO]	Lista que la pantalla muestra.
error	str	Mensaje de validación del formulario (vacío = sin error).
dialog_open	bool	Controla si el diálogo está visible.
editing_id	int	0 = se está creando una tarea nueva; mayor que 0 = se edita esa tarea.
form_title, form_description, form_priority, form_due_date	str	Valores iniciales del formulario.

Eventos del estado

Evento	Cuándo se dispara	Qué hace
load()	Al cargar la página (on_load).	Lee todas las tareas de la base de datos y las guarda en tasks.
set_dialog_open(value)	Cuando el diálogo se abre o se cierra (Esc, clic fuera).	Actualiza dialog_open y, al cerrar, borra el error.
open_new()	Botón New task.	Vacía el formulario, pone editing_id = 0 y abre el diálogo.
open_edit(task_id)	Icono del lápiz.	Busca la tarea en la lista en memoria, copia sus valores a los campos form_* y abre el diálogo.
save(form)	Envío del formulario.	Valida con Pydantic y crea o actualiza según editing_id. Si hay error, lo muestra y deja el diálogo abierto.
toggle(task_id, done)	Casilla de la tarea.	Actualiza solo done y recarga la lista.
delete(task_id)	Icono de la papelera.	Elimina la tarea y recarga la lista.

Análisis del evento save

1. Reflex entrega el formulario como un diccionario `form` cuyas claves son los atributos `name=` de cada campo (title, description, priority, due_date).
2. Se construye `data` limpiando espacios con `.strip()` y convirtiendo la fecha vacía en `None`.
3. Si `editing_id` es distinto de 0, se llama a `update_task` con un `TaskUpdateDTO`; si no, a `create_task` con un `TaskCreateDTO`.
4. Al construir el DTO, Pydantic valida. Si algo falla (p. ej. título vacío) salta `ValidationError`: se guarda el primer mensaje en `self.error` y se hace `return`, así el diálogo sigue abierto.
5. Si todo va bien: se limpia el error, se cierra el diálogo y se recarga la lista con `list_tasks()`.

Paso 9: La interfaz de usuario (pages/index.py)

Archivo: crud_agenda_app/pages/index.py

```
import reflex as rx

from crud_agenda_app.db.schemas.task import TaskDTO
from crud_agenda_app.states.task_state import TaskState

PRIORITY_COLOR = {"low": "gray", "medium": "amber", "high": "red"}


def task_row(task: TaskDTO) -> rx.Component:
    return rx.card(
        rx.hstack(
            rx.checkbox(
                checked=task.done,
                on_change=lambda v: TaskState.toggle(task.id, v),
            ),
            rx.vstack(
                rx.text(
                    task.title,
                    weight="bold",
                    text_decoration=rx.cond(task.done, "line-through", "none"),
                ),
                rx.cond(task.description != "", rx.text(task.description, size="2", color_scheme="gray")),
                rx.hstack(
                    rx.badge(
```

```
                        task.priority,
                        color_scheme=rx.match(
                            task.priority,
                            ("high", "red"),
                            ("medium", "amber"),
                            "gray",
                        ),
                    ),
                    rx.cond(task.due_date != "", rx.badge(task.due_date, variant="outline")),
                    spacing="2",
                ),
                spacing="1",
                align="start",
                flex="1",
            ),
            rx.icon_button(
                rx.icon("pencil"),
                variant="ghost",
                on_click=TaskState.open_edit(task.id),
            ),
            rx.icon_button(
                rx.icon("trash-2"),
                variant="ghost",
                color_scheme="red",
                on_click=TaskState.delete(task.id),
            ),
            align="center",
            spacing="3",
            width="100%",
        ),
        width="100%",
    )


def task_dialog() -> rx.Component:
    return rx.dialog.root(
        rx.dialog.content(
            rx.dialog.title(rx.cond(TaskState.editing_id > 0, "Edit task", "New task")),
            rx.form(
                rx.vstack(
                    rx.input(
                        name="title",
                        placeholder="Title",
                        default_value=TaskState.form_title,
                        width="100%",
                    ),
                    rx.text_area(
                        name="description",
                        placeholder="Description",
                        default_value=TaskState.form_description,
                        width="100%",
                    ),
                    rx.select(
                        ["low", "medium", "high"],
                        name="priority",
                        default_value=TaskState.form_priority,
                        width="100%",
                    ),
                    rx.input(
                        name="due_date",
                        type="date",
                        default_value=TaskState.form_due_date,
                        width="100%",
                    ),
                    rx.cond(
                        TaskState.error != "",
                        rx.callout(TaskState.error, color_scheme="red", icon="triangle-alert"),
                    ),
                    rx.hstack(
                        rx.dialog.close(rx.button("Cancel", variant="soft", color_scheme="gray", type="button")),
                        rx.button("Save", type="submit"),
                        justify="end",
                        width="100%",
                    ),
```

```
                    spacing="3",
                ),
                on_submit=TaskState.save,
                reset_on_submit=False,
            ),
        ),
        open=TaskState.dialog_open,
        on_open_change=TaskState.set_dialog_open,
    )


def index_page() -> rx.Component:
    return rx.container(
        rx.vstack(
            rx.hstack(
                rx.heading("Agenda", size="7"),
                rx.spacer(),
                rx.button(rx.icon("plus"), "New task", on_click=TaskState.open_new),
                width="100%",
                align="center",
            ),
            rx.cond(
                TaskState.tasks.length() > 0,
                rx.vstack(rx.foreach(TaskState.tasks, task_row), spacing="2", width="100%"),
                rx.text("No tasks yet.", color_scheme="gray"),
            ),
            task_dialog(),
            spacing="4",
            width="100%",
        ),
        size="2",
        padding_y="2em",
    )
```

Importante: reglas especiales de Reflex

Los valores del estado (**TaskState.tasks**, **task.done**...) no son valores reales de Python en el momento de construir la interfaz, sino **variables reactivas** que se evalúan en el navegador. Por eso no se pueden usar **if**, **for** ni **match** de Python con ellas; hay que usar sus equivalentes:

En Python normal	En Reflex	Aparece en el código
if / else	rx.cond(condición, si_cierto, si_falso)	Tachar el título, mostrar la descripción, lista vacía
for item in lista	rx.foreach(lista, función)	rx.foreach(TaskState.tasks, task_row)
match / case	rx.match(valor, (caso, resultado), ..., defecto)	Color del badge según la prioridad

Las tres funciones del archivo

task_row(task): dibuja una tarea dentro de un **rx.card**, con un **rx.hstack** (fila) que contiene:

- Una casilla `rx.checkbox` cuyo `checked` refleja `task.done`. Su `on_change` usa una `lambda v: ...` porque Reflex entrega el nuevo valor (True/False) y hay que pasarlo junto al id a `TaskState.toggle`.
- Una columna con el título (tachado si está hecha), la descripción (solo si no está vacía) y dos etiquetas: la prioridad, coloreada con rx.match (high = rojo, medium = ámbar, resto = gris), y la fecha límite si existe.
- Un botón con icono de lápiz (`open_edit`) y otro con icono de papelera en rojo (`delete`), ambos enviando `task.id`.

task_dialog(): el diálogo modal para crear y editar.

- Es un diálogo **controlado**: `open=TaskState.dialog_open` decide si se ve y `on_open_change=TaskState.set_dialog_open` avisa al estado cuando el usuario lo cierra.
- El título cambia con `rx.cond(TaskState.editing_id > 0, "Edit task", "New task")`.
- Dentro hay un `rx.form` con los campos `title`, `description`, `priority` (lista desplegable) y `due_date` (selector de fecha). Cada uno tiene `name=` (la clave que llegará a `save`) y `default_value` enlazado al estado, que es lo que rellena el formulario al editar.
- `rx.callout` muestra `TaskState.error` solo si no está vacío.
- El botón Cancel va envuelto en `rx.dialog.close` y tiene `type="button"` para que no envíe el formulario; el botón Save es `type="submit"`.
- `on_submit=TaskState.save` conecta el envío con el evento, y `reset_on_submit=False` evita que se vacíe el formulario si hay un error de validación.

index_page(): la página completa.

- Un contenedor con una cabecera (título "Agenda" y el botón "New task", que dispara open_new).
- `rx.cond(TaskState.tasks.length() > 0, ...)`: si hay tareas muestra la lista con `rx.foreach`; si no, el texto "No tasks yet.".
- Incluye task_dialog() al final; el diálogo permanece oculto hasta que dialog_open sea True.

Paso 10: Punto de entrada de la aplicación (crud_agenda_app.py)

Archivo: crud_agenda_app/crud_agenda_app.py

```
import reflex as rx

from crud_agenda_app.db.base import init_db
from crud_agenda_app.pages.index import index_page
from crud_agenda_app.states.task_state import TaskState

init_db()  # prototype only; replace with Alembic migrations later

app = rx.App(
    theme=rx.theme(appearance="light", accent_color="indigo", radius="medium"),
)

app.add_page(index_page, route="/", title="Agenda", on_load=TaskState.load)
```

- `init_db()`: se ejecuta al importar el módulo, antes de arrancar. Crea la tabla `tasks` si no existe.
- `rx.App(theme=rx.theme(...))`: crea la aplicación con tema claro, color de acento índigo y esquinas de radio medio.
- `app.add_page(...)`: registra la página `index_page` en la ruta `/` con título de pestaña "Agenda". El parámetro `on_load=TaskState.load` hace que cada vez que se abra la página se carguen las tareas de la base de datos.

Paso 11: Ejecutar y probar la aplicación

```
uv sync                # instala las dependencias desde uv.lock
uv run reflex run      # compila el frontend y arranca frontend + backend
```

- La primera ejecución tarda más: Reflex instala los paquetes del frontend y genera la carpeta **.web**.
- Abre **http://localhost:3000** en el navegador (el frontend). El servidor Python escucha en el puerto **8000**, que es el valor de api_url.
- En modo desarrollo, al guardar un archivo .py la aplicación se recarga sola.

Lista de comprobación

1. Pulsa **New task**, escribe un título y pulsa **Save**: la tarea aparece arriba de la lista.
2. Intenta guardar con el título vacío: debe aparecer un mensaje de error rojo y el diálogo sigue abierto.
3. Pulsa el lápiz, cambia la prioridad y guarda: el badge cambia de color.
4. Marca la casilla: el título queda tachado. Desmárcala: vuelve a la normalidad.
5. Pulsa la papelera: la tarea desaparece. Recarga la página: los cambios persisten (están en SQLite).

5. Cómo viaja la información: flujo de cada operación

Este resumen une todas las piezas. Cada fila es el recorrido completo desde la acción del usuario hasta la base de datos y de vuelta a la pantalla.

Operación	Acción del usuario	Evento	Servicio	Resultado en la BD
Read	Abrir la página	TaskState.load	list_tasks()	SELECT ... ORDER BY created_at DESC
Create	New task, rellenar y Save	open_new, luego save (editing_id = 0)	create_task(TaskCreateDTO)	INSERT INTO tasks
Update	Lápiz, modificar y Save	open_edit, luego save (editing_id > 0)	update_task(id, TaskUpdateDTO)	UPDATE tasks SET ...
Update (parcial)	Marcar la casilla	toggle(id, done)	update_task(id, TaskUpdateDTO(done=...))	UPDATE tasks SET done = ?
Delete	Pulsar la papelera	delete(id)	delete_task(id)	DELETE FROM tasks WHERE id = ?

Ejemplo completo: crear una tarea

1. El usuario pulsa **New task**. El navegador envía el evento **open_new** al servidor.
2. El estado vacía el formulario, pone **editing_id = 0** y **dialog_open = True**. Reflex devuelve el cambio y el diálogo aparece.
3. El usuario escribe "Comprar pan", elige prioridad **high** y pulsa **Save**. El formulario se envía como diccionario a **save**.
4. **save** crea **TaskCreateDTO(title="Comprar pan", priority="high", ...)**. Pydantic valida los datos.
5. **create_task** abre una transacción, inserta la fila (SQLite asigna el id y la fecha de creación) y hace commit.
6. **save** cierra el diálogo y vuelve a pedir **list_tasks()**. El estado recibe la lista nueva.
7. Reflex envía el cambio de **tasks** al navegador y **rx.foreach** dibuja una tarjeta más.

6. Observaciones y mejoras recomendadas

Al revisar el código se encontraron algunos puntos que conviene conocer. Ninguno impide que la aplicación funcione en su estado actual, pero conviene corregirlos.

#	Dónde	Observación	Sugerencia
1	config.py	ROOT = Path(__file__).resolve().parent.parent apunta a la carpeta **superior** a la raíz del proyecto, porque config.py ya está en la raíz. El .env se buscaría fuera del proyecto.	Usar .parent una sola vez, o simplemente env_file=".env".
2	db/models/__init__.py	Está vacío. init_db() hace "from ... import models" esperando que registre Task, pero no importa nada. Funciona solo porque task_service ya importó el modelo antes de llamar a init_db().	Añadir **from .task import Task** en db/models/__init__.py.
3	states/task_state.py	open_edit asigna task.due_date (un objeto date o None) a form_due_date, que está declarada como str.	Usar **str(task.due_date) if task.due_date else ""**.
4	pyproject.toml	El script crud-agenda-app apunta a crud_agenda_app:main, pero el __init__.py del paquete está vacío y no define main.	Definir main() o eliminar la sección.
5	pages/index.py	PRIORITY_COLOR está definido pero no se usa (la página usa rx.match).	Eliminarlo o usarlo para evitar duplicar la lógica.
6	task_state.py y service	toggle() y save() no capturan LookupError si la tarea fue eliminada desde otra pestaña.	Capturar la excepción y recargar la lista.
7	db/base.py	create_all es válido para prototipos, pero no gestiona cambios del esquema.	Adoptar Alembic cuando cambie el modelo.
8	Repositorio	Las carpetas .web, .states y el archivo .db son generados.	Añadirlos a .gitignore.
9	src/	Carpeta vacía sobrante de la plantilla de uv.	Eliminarla.
10	Interfaz	Los textos están en inglés (New task, Edit task, No tasks yet).	Traducirlos si la aplicación es para público hispanohablante.

Ideas para ampliar la aplicación

- Filtros por estado (pendientes / hechas) y por prioridad; búsqueda por texto.
- Ordenar por fecha límite y resaltar las tareas vencidas.
- Confirmación antes de eliminar.
- Usuarios y autenticación (aquí servirían secret_key y env = "prod" de la configuración).
- Pruebas automáticas del servicio con una base SQLite en memoria (sqlite://).

7. Glosario rápido

Término	Significado
CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre datos.
ORM	Object-Relational Mapper. Permite trabajar con tablas como si fueran clases de Python (SQLAlchemy).
Engine	Objeto de SQLAlchemy que mantiene la conexión con la base de datos.
Sesión	Unidad de trabajo con la base de datos; agrupa operaciones en una transacción.
Transacción / commit / rollback	Conjunto de cambios que se guardan todos juntos (commit) o se descartan todos (rollback).
DTO	Data Transfer Object: objeto simple para mover datos entre capas.
Estado (State)	Datos de la aplicación en el servidor que Reflex sincroniza con la pantalla.
Evento	Método del estado que se ejecuta como respuesta a una acción del usuario.
Variable reactiva	Valor del estado que, al cambiar, actualiza automáticamente la interfaz.
Migración	Script versionado que modifica el esquema de la base de datos (Alembic).